개요 및 아키텍처

목차

- halo2-lib란 무엇인가
- 프로젝트 구조
- 핵심 설계 원칙
- 아키텍처 다이어그램
- 관련 문서

halo2-lib란 무엇인가

halo2-lib는 Axiom Crypto에서 개발한 Halo2 영지식증명 시스템을 위한 고수준 라이브러리입니다. Halo2는 PLONKish 산술화(arithmetization)를 사용하는 영지식증명 시스템으로, 신뢰할 수 있는 설정(trusted setup)이 필요하지만 범용적이고 효율적인 증명 생성이 가능합니다.

주요 특징

halo2-lib는 다음과 같은 특징을 제공합니다:

1. **간소화된 회로 작성 API**: 복잡한 Halo2의 저수준 API를 추상화하여 단일 advice 컬럼과 selector 컬럼에 대한 제약 조건 선언으로 회로 프로그래밍을 단순화합니다.
2. **가상 영역 관리**: 물리적 Halo2 회로로 매핑되기 전에 가상 영역(virtual region)에서 witness 생성을 수행하여 회로 구성의 유연성을 제공합니다.
3. **자동 회로 튜닝**: 회로 파라미터(행과 열의 수)를 자동으로 최적화하여 증명 시간과 검증 비용 간의 트레이드오프를 조정할 수 있습니다.

4. **멀티스레드 witness 생성**: 병렬 witness 생성을 지원하여 대규모 회로의 증명 생성 속도를 향상시킵니다.

5. **타원곡선 암호 라이브러리**: secp256k1, BN254 등 다양한 타원곡선에 대한 최적화된 암호학적 연산을 제공합니다.

사용 사례

halo2-lib는 다음과 같은 프로젝트에서 사용되고 있습니다:

- **Axiom**: 이더리움 온체인 데이터에 대한 증명
- **zkEmail**: 이메일 도메인의 신뢰 가정으로 이메일에 대한 사실 증명
- **zkEVM**: Scroll의 zkEVM 서명 검증 및 트랜잭션 회로
- **WebAuthn**: WebAuthn 증명 및 검증
- **Spectre**: Beacon 체인 헤더 검증

프로젝트 구조

halo2-lib는 Rust 워크스페이스로 구성되어 있으며, 두 개의 주요 크레이트로 이루어져 있습니다.

워크스페이스 구조

```
halo2-lib/
├── Cargo.toml # 워크스페이스 설정
├── README.md # 프로젝트 문서
├── halo2-base/ # 기본 회로 구성 라이브러리
| ├── Cargo.toml
| ├── src/
| | ├── lib.rs # 핵심 타입 정의
```

```
| | ├── gates/ # 게이트 구현
| | ├── virtual_region/ # 가상 영역 관리
| | ├── poseidon/ # Poseidon 해시
| | ├── safe_types/ # 타입 안전성
| | └── utils/ # 유틸리티
| ├── examples/ # 사용 예제
| └── benches/ # 벤치마크
└── halo2-ecc/ # 타원곡선 암호 라이브러리
├── Cargo.toml
├── src/
| ├── lib.rs
| ├── bigint/ # 큰 정수 연산
| ├── fields/ # 소수체 연산
| ├── ecc/ # 타원곡선 점 연산
| ├── secp256k1/ # secp256k1 특화
| └── bn254/ # BN254 특화
└── benches/ # 벤치마크
```

Cargo.toml 구성

워크스페이스의 `Cargo.toml` 은 다음과 같이 구성되어 있습니다:

```
[workspace]
members = ["halo2-base", "halo2-ecc"]
```

```
resolver = "2"



[profile.release]

opt-level = 3

debug = false

debug-assertions = false

lto = "fat"

panic = "unwind"

incremental = false
```

주요 프로파일:

- **dev**: 빠른 컴파일과 디버깅을 위한 개발 모드
- **local**: dev보다 최적화되었지만 release보다 빠른 컴파일
- **release**: 최대 성능을 위한 프로덕션 모드
- **flamegraph**: 성능 프로파일링을 위한 모드

halo2-base: 기본 회로 구성 라이브러리

halo2-base는 halo2-lib의 핵심 크레이트로, 회로 작성을 위한 기본 API와 데이터 구조를 제공합니다.

주요 기능

1. **간소화된 게이트 API**: `GateInstructions` 와 `RangeInstructions` 트레이트를 통해 기본 산술 연산과 범위 검사를 제공합니다.
2. **가상 셀 관리**: `Context` , `AssignedValue` , `QuantumCell` 등의 타입을 통해 회로의 실행 추적(execution trace)을 관리합니다.

3. **회로 빌더**: `BaseCircuitBuilder` 를 통해 회로를 구성하고 최적화합니다.

의존성

```
[dependencies]

# Halo2 백엔드 (둘 중 하나 선택)

halo2_proofs_axiom = { version = "0.4", optional = true } # Axiom 최적화 버전

halo2_proofs = { git = "...", optional = true } # PSE 안정 버전


# 핵심 의존성

itertools = "0.11"

num-bigint = { version = "0.4", features = ["rand"] }

rayon = "1.8" # 병렬 처리

serde = { version = "1.0", features = ["derive"] }
```

모듈 구조

- **gates**: 커스텀 게이트 구현
- `flex_gate` : 기본 산술 게이트
- `range` : 범위 검사 게이트
- `circuit` : 회로 빌더와 설정
- **virtual_region**: 가상 영역 관리 시스템
- `manager` : VirtualRegionManager 트레이트
- `copy_constraints` : equality constraint 관리

- `lookups` : lookup argument 관리
- **poseidon**: Poseidon 해시 함수 구현
- **safe_types**: 타입 안전성을 위한 래퍼
- **utils**: 유틸리티 함수

halo2-ecc: 타원곡선 암호 라이브러리

halo2-ecc는 halo2-base를 기반으로 타원곡선 암호학적 연산을 제공하는 라이브러리입니다. 특히 증명 시스템의 스칼라 필드보다 큰 베이스 필드를 가진 타원곡선을 지원합니다.

주요 기능

1. **큰 정수 연산**: 네이티브 필드보다 큰 정수에 대한 최적화된 연산
2. **소수체 연산**: 확장 필드(Fp2, Fp12) 포함
3. **타원곡선 연산**: 점 덧셈, 배가, 스칼라 곱셈, MSM
4. **암호학적 프로토콜**: ECDSA, Schnorr 서명, Pairing

의존성

```
[dependencies]

halo2-base = { version = "=0.4.1", path = "../halo2-base" }

num-bigint = { version = "0.4", features = ["rand"] }

rayon = "1.8"

serde = { version = "1.0", features = ["derive"] }
```

모듈 구조

- **bigint**: 큰 정수 산술 연산
- `mul_no_carry`, `carry_mod` : carry 최적화
- `big_less_than`, `big_is_zero` : 비교 연산
- **fields**: 소수체 연산
- `fp` : 기본 소수체
- `fp2` : 이차 확장 필드
- `fp12` : 12차 확장 필드 (pairing용)
- **ecc**: 타원곡선 점 연산
- `fixed_base`, `pippenger` : MSM 알고리즘
- `ecdsa` : ECDSA 서명 검증
- **secp256k1**: secp256k1 곡선 특화 구현
- **bn254**: BN254 곡선 특화 구현
- `pairing` : 최적 Ate pairing
- `final_exp` : Final exponentiation

핵심 설계 원칙

1. 가상 영역 관리 (Virtual Region Management)

halo2-lib의 가장 중요한 설계 원칙은 **가상 영역 관리**입니다. 이는 witness 생성과 물리적 회로 할당을 분리하여 다음과 같은 이점을 제공합니다:

가상 영역의 개념

- **가상 컬럼**: `Context` 는 제약 조건과 witness 값을 저장하는 "가상 컬럼"을 나타냅니다.
- **물리적 매핑**: witness 생성 후, 가상 컬럼들은 실제 Halo2 회로의 물리적 컬럼으로 매핑됩니다.

- **유연한 재배치**: 동일한 회로 로직을 다양한 물리적 구성(행 vs 열의 수)으로 배치할 수 있습니다.

VirtualRegionManager 트레이트

```
pub trait VirtualRegionManager<F: Field> {
type Config: Clone;
type Assignment = ();
fn assign_raw(&self, config: &Self::Config, region: &mut Region<F>) -> Self::As
}
```

모든 가상 영역 관리자는 이 트레이트를 구현하여:

1. 자신만의 방식으로 가상 영역을 관리
2. 결정론적으로 가상 영역을 물리적 영역으로 매핑

주요 가상 영역 관리자

1. **SinglePhaseCoreManager**:
- `FlexGateConfig` 와 연관
- 단일 challenge phase의 가상 컬럼 관리
- 목표 행 수에 맞춰 가상 컬럼을 물리적 컬럼으로 chunking
2. **CopyConstraintManager**:
- 전역 equality constraint 관리
- `AssignedValue` 간의 참조 추적
- 상수 값 중복 제거 및 fixed 컬럼 할당
3. **LookupAnyManager**:
- Lookup argument 처리

- 특수 advice 컬럼에 lookup 값 복사
- 비구조적 lookup 위치 지원

2. BaseCircuitBuilder의 중요성

`BaseCircuitBuilder` 는 halo2-lib의 핵심 회로 빌더로, 다음과 같은 역할을 수행합니다:

회로 구성 관리

```
// 회로 빌더 생성
let k = 10; // 2^10 = 1024 rows
let mut builder = BaseCircuitBuilder::new(false).use_k(k);
builder.set_lookup_bits(k - 1); // lookup 테이블 설정

// Context 생성 및 연산 수행
let ctx = builder.main(0); // phase 0
let gate = GateChip::default();
let a = ctx.load_witness(F::from(5));
let b = ctx.load_witness(F::from(3));
let c = gate.mul(ctx, a, b);

// 회로 파라미터 최적화
builder.calculate_params(Some(9));
```

주요 기능

1. **회로 크기 설정**: `use_k(k)` 로 2^k 행을 가진 회로 생성

2. **Lookup 테이블 설정**: `set_lookup_bits()` 로 범위 검사 테이블 크기 지정

3. **Context 관리**: 각 challenge phase별 Context 생성

4. **자동 최적화**: `calculate_params()` 로 최적 컬럼 수 계산

5. **증명 생성**: MockProver 또는 실제 증명 생성 지원

설정 가능한 회로 (Configurable Circuits)

halo2-base로 작성된 회로의 특별한 특징은 **동일한 총 셀 수를 유지하면서 행과 열의 수를 다르게 구성**할 수 있다는 것입니다:

- **더 많은 행, 적은 열**: 온체인 검증 가스 비용 감소, 증명 시간 증가
- **적은 행, 더 많은 열**: 증명 시간 감소, 검증 비용 증가

이는 `BaseCircuitParams` 를 통해 제어됩니다:

```
pub struct BaseCircuitParams {
pub k: usize, // 회로 크기: 2^k rows
pub num_advice_per_phase: Vec<usize>, // phase별 advice 컬럼 수
pub num_fixed: usize, // fixed 컬럼 수
pub num_lookup_advice_per_phase: Vec<usize>, // lookup advice 컬럼 수
pub lookup_bits: Option<usize>, // lookup 테이블 비트 수
pub num_instance_columns: usize, // public instance 컬럼 수
}
```

3. 단순화된 게이트 전략

halo2-lib는 **Vertical Gate Strategy**를 사용하여 회로 작성을 단순화합니다:

FlexGate: 범용 산술 게이트

단일 제약 조건으로 다양한 연산 표현:

```
q * (a + b * c - d) = 0
```

이 하나의 게이트로 다음을 모두 구현:

- 덧셈: `a + b` (c=0, d=out)
- 곱셈: `a * b` (a=0, c=b, d=out)
- 내적: $\Sigma(a_i * b_i)$ (여러 행 사용)

RangeGate: 범위 검사 게이트

Lookup 테이블을 사용한 효율적인 범위 검사:

- `range_check(a, bits)` : a < 2^bits 검증
- `check_less_than(a, b, bits)` : a < b 검증

4. 멀티스레드 Witness 생성

대규모 회로의 성능을 위해 병렬 witness 생성 지원:

1. **병렬 Context 생성**: 각 스레드가 독립적인 Context에서 작업
2. **결정론적 병합**: Context들을 결정론적 순서로 병합
3. **Copy Constraint 관리**: `CopyConstraintManager` 가 스레드 간 참조 추적

아키텍처 다이어그램

전체 시스템 아키텍처

```
┌──────────────────────────────────────────────────────────────────────┐
| halo2-lib |
```

```
├──────────────────────────────────────────────────────────────┤
| |
| ┌──────────────────────┐ ┌──────────────────────┐ |
| | halo2-base | | halo2-ecc | |
| | | | | |
| | ┌────────────────┐ | | ┌────────────────┐ | |
| | | BaseCircuit | | | | bigint | | |
| | | Builder | | | | - mul_no_carry | | |
| | └────────────────┘ | | | - carry_mod | | |
| | | | | └────────────────┘ | |
| | ┌────────────────┐ | | | |
| | | FlexGate | | | ┌────────────────┐ | |
| | | - add, mul | | | | fields | | |
| | | - inner_product| | | | - fp, fp2, fp12| | |
| | └────────────────┘ | | └────────────────┘ | |
| | | | | |
| | ┌────────────────┐ | | ┌────────────────┐ | |
| | | RangeGate | | | | ecc | | |
| | | - range_check | | | | - ec_add | | |
| | | - less_than | | | | - scalar_mul | | |
| | └────────────────┘ | | | - MSM | | |
| | | | | └────────────────┘ | |
| | ┌────────────────┐ | | | |
```

```
| | | Virtual Region | | | ┌──────────────┐ | |
| | | Managers | | | | secp256k1 | | |
| | | - Core | | | | - ECDSA | | |
| | | - CopyConstr | | | └──────────────┘ | |
| | | - Lookup | | | | |
| | └──────────────┘ | | ┌──────────────┐ | |
| └──────────────────────┘ | | bn254 | | |
| | | - Pairing | | |
| | | - final_exp | | |
| | └──────────────┘ | |
| └──────────────────────┘ |
└──────────────────────────────────────────────────────────────┘
|
▼
┌──────────────────────────────────────────────────────────────┐
| Halo2 Backend |
| (halo2_proofs_axiom 또는 halo2_proofs) |
└──────────────────────────────────────────────────────────────┘
```

가상 영역에서 물리적 회로로의 매핑

```
Witness 생성 단계 (가상 영역)
┌──────────────────────────────────────────┐
```

```
| Thread 1 Thread 2 Thread 3 |

| ┌────────┐ ┌────────┐ ┌────────┐|

| |Context | |Context | |Context ||

| | (C1) | | (C2) | | (C3) ||

| | | | | | ||

| | cell 0 | | cell 0 | | cell 0 ||

| | cell 1 | | cell 1 | | cell 1 ||

| | cell 2 | | cell 2 | | cell 2 ||

| | ... | | ... | | ... ||

| └────────┘ └────────┘ └────────┘|

└──────────────────────────────────────┘

|

▼ 병합 (결정론적)

┌──────────────────────────────────────┐

| 단일 가상 컬럼 |

| ┌──────────────────────────────┐ |

| | C1 cells → C2 cells → C3 cells | |

| └──────────────────────────────┘ |

└──────────────────────────────────────┘

|

▼ Chunking (break points 기반)

┌──────────────────────────────────────┐

| 물리적 Halo2 회로 |
```

```
│ ┌─────────┐ ┌─────────┐ ┌─────────┐ │
│ │ Advice │ │ Advice │ │ Advice │ │
│ │ Col 0 │ │ Col 1 │ │ Col 2 │ │
│ │ │ │ │ │ │ │
│ │ 2^k │ │ 2^k │ │ 2^k │ │
│ │ rows │ │ rows │ │ rows │ │
│ └─────────┘ └─────────┘ └─────────┘ │
└──────────────────────────────────────┘
```

회로 구성 흐름

```
1. 초기화
┌──────────────────────────┐
│ BaseCircuitBuilder::new()│
│ .use_k(k) │
│ .set_lookup_bits(bits) │
└──────────────────────────┘
│
▼
2. Witness 생성
┌──────────────────────────┐
│ let ctx = builder.main(0)│
│ let gate = GateChip::.. │
```

```
| let a = ctx.load_witness |
| let c = gate.mul(ctx,..) |
└──────────────────────────┘
|
▼
3. 회로 최적화
┌──────────────────────────┐
| builder.calculate_params |
| - break points 계산 |
| - 최적 컬럼 수 결정 |
└──────────────────────────┘
|
▼
4. 증명 생성
┌──────────────────────────┐
| MockProver::run() |
| 또는 |
| create_proof() |
└──────────────────────────┘
```

관련 문서

- [02-핵심-개념.md](): Context, AssignedValue, QuantumCell 상세 설명

- 03-회로-구성-프로세스.md: BaseCircuitBuilder 사용법
- 04-제약-조건-시스템.md: FlexGate와 RangeGate 구현
- 05-가상-영역-관리.md: Virtual Region Manager 상세
- 06-타원곡선-암호.md: halo2-ecc ECC 연산
- 07-실전-예제-분석.md: ECDSA, Pairing 구현 예제

다음 단계

이제 halo2-lib의 전체 구조를 이해했다면, 다음 문서에서 핵심 데이터 타입들을 상세히 학습하세요:

→ **02. 핵심 개념**: Context, AssignedValue, QuantumCell, ContextCell의 정의와 사용법

참고 자료

소스 파일

- `Cargo.toml` : 워크스페이스 설정
- `halo2-base/src/lib.rs` : 핵심 타입 정의
- `halo2-base/src/gates/circuit/mod.rs` : BaseCircuitBuilder
- `halo2-base/src/virtual_region/manager.rs` : VirtualRegionManager 트레이트
- `halo2-ecc/src/lib.rs` : ECC 라이브러리 진입점

외부 링크

- Halo2 공식 문서
- Axiom 문서
- halo2-lib GitHub
- Rust 문서